IluminaBH

Arquitetura do Software Baseado em IA

Sistema Inteligente de Análise da Iluminação Pública de Belo Horizonte

Time de Engenharia de IA | PUC Minas | Entrega: 06/12/2026

1. Introdução

Este documento descreve a arquitetura do software baseado em IA que o Time de Engenharia de IA entregará até 06/12/2026. O IluminaBH é uma aplicação web, executada localmente, que integra dados públicos de iluminação pública de Belo Horizonte, trata e armazena esses dados, permite explorá-los por mapas, filtros, indicadores e relatórios e usa aprendizado de máquina não supervisionado para identificar perfis semelhantes da infraestrutura.

Tudo o que está definido aqui segue a Lista de Requisitos do projeto (RF01–RF15, RNF01–RNF12, RD01–RD06 e RIA01–RIA10). Cada etapa da arquitetura indica os requisitos que atende, e a Seção 8 reúne a rastreabilidade completa. O documento apresenta: (i) a Figura 1, com todas as atividades do sistema; (ii) a explicação de cada etapa; (iii) as técnicas e algoritmos de IA escolhidos; e (iv) o papel das bases do Portal de Dados Abertos da Prefeitura de Belo Horizonte (PBH) na solução.

Delimitação. A IA identifica padrões nos dados. Ela não detecta lâmpadas queimadas, não prevê falhas, consumo ou manutenção e não avalia a qualidade da iluminação (RIA01). Um perfil encontrado pelo modelo é uma estrutura nos dados, nunca um diagnóstico oficial nem um julgamento de "bom" ou "ruim".

2. Visão geral da arquitetura

A Figura 1 mostra o fluxo completo, da fonte de dados ao usuário. As cores indicam a natureza de cada bloco: azul para a fonte de dados, verde para engenharia de dados, laranja para o pipeline de aprendizado de máquina e roxo para API e aplicação web. Os círculos numerados correspondem às etapas descritas na Seção 3. Os blocos tracejados são saídas ou observações de uma etapa.

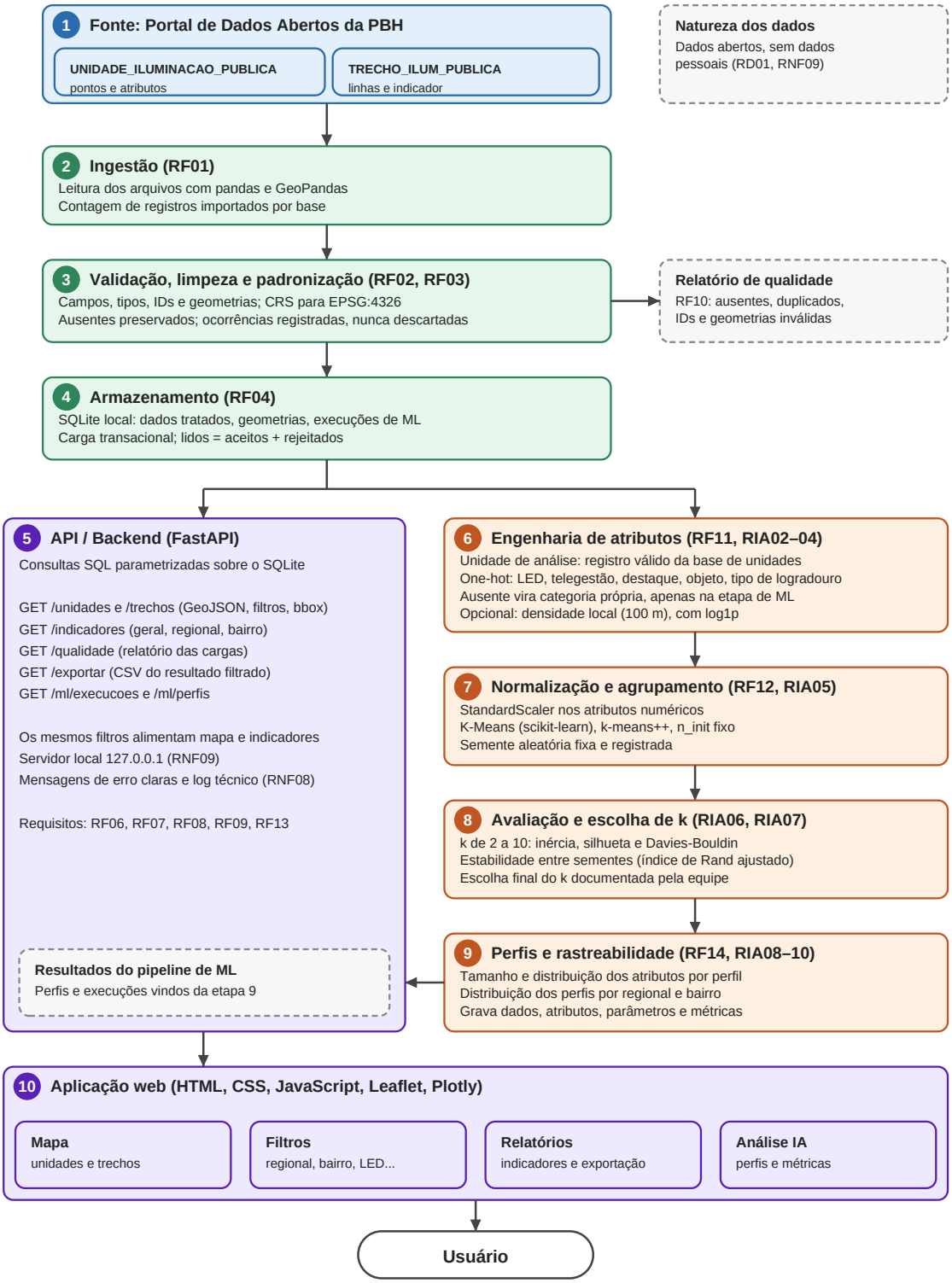

Figura 1. Arquitetura do IluminaBH: ingestão, tratamento, armazenamento, API, pipeline de ML e aplicação web, executados localmente.

3. Descrição das etapas

Cada etapa abaixo corresponde a um número da Figura 1.

Etapa 1 – Fonte de dados: Portal de Dados Abertos da PBH

O sistema parte de duas bases públicas: **UNIDADE_ILUMINACAO_PUBLICA** (pontos, com identificação, iluminação de destaque, objeto iluminado, modernização para LED, telegestão, tipo e nome do logradouro, bairro, regional e geometria) e **TRECHO_ILUM_PUBLICA** (linhas, com identificação do trecho, identificação da base, indicador disponível e geometria). Os arquivos são baixados do portal e guardados sem alteração em uma pasta de dados brutos. Depois da instalação, o sistema não precisa de internet para operar com eles.

Requisitos atendidos: RD01, RD02, RD03, RNF01, RNF09

Etapa 2 – Ingestão

Um módulo Python lê os arquivos (CSV e/ou GeoJSON) com pandas e GeoPandas, confere o formato e as colunas esperadas e informa a quantidade de registros importados de cada base. A ingestão não corrige nada: apenas carrega os dados para a etapa seguinte.

Requisitos atendidos: RF01, RNF08

Etapa 3 – Validação, limpeza e padronização

Regras documentadas e cobertas por testes automatizados (pytest) verificam campos obrigatórios, tipos, identificadores ausentes ou duplicados, categorias fora do domínio da fonte e geometrias (tipo ponto ou linha, validade, localização no município). A limpeza remove espaços excedentes e padroniza caixa e formatos de texto. Valores ausentes permanecem ausentes: o sistema não lhes atribui um significado que a fonte não define. Registros com problema são registrados com o motivo, não são removidos em silêncio. As geometrias são convertidas para o sistema de referência adotado (EPSG:4326). O resultado alimenta o **relatório de qualidade dos dados**.

Requisitos atendidos: RF02, RF03, RF10, RD04, RD05, RD06, RNF07, RNF08, RNF11

Etapa 4 – Armazenamento

Os registros aprovados são gravados em um banco SQLite local, em carga transacional. Em cada carga, a quantidade de registros lidos deve ser igual à de aceitos mais rejeitados, o que permite verificar que não houve perda ou duplicação (RNF07). O banco guarda também as ocorrências de qualidade e as execuções do modelo de IA. Se as consultas geográficas exigirem mais recursos, o PostgreSQL/PostGIS será avaliado como evolução controlada.

Requisitos atendidos: RF04, RNF01, RNF02, RNF07

Etapa 5 – API / Backend

Um servidor FastAPI, acessível apenas em 127.0.0.1, expõe os dados por endpoints. Os filtros (regional, bairro, tipo de logradouro, LED, telegestão, objeto iluminado e iluminação de destaque) viram consultas SQL parametrizadas, e os mesmos filtros alimentam o mapa e os indicadores. A API calcula os indicadores (totais, percentuais de LED e telegestão, unidades de destaque, por regional e por bairro), gera a exportação em CSV e entrega as camadas em GeoJSON limitadas pela área visível do mapa, para manter a interação fluida. Também serve ao front-end os perfis e as execuções do modelo.

Requisitos atendidos: RF06, RF07, RF08, RF09, RF13, RNF03, RNF08, RNF09

Etapa 6 – Engenharia de atributos

A unidade de análise é o registro válido da base de unidades (RIA02). Os trechos entram como informação geográfica complementar. Os atributos do modelo são: modernização para LED, telegestão, iluminação de destaque, objeto iluminado e tipo de logradouro, todos categóricos, convertidos por codificação one-hot. Como candidato opcional, um atributo numérico de densidade local (unidades em um raio de 100 m, com log1p), que só entra se a experimentação o justificar. Identificadores técnicos não são atributos (RIA03). Bairro e regional *não* entram no modelo: são usados depois, para descrever onde cada perfil aparece, evitando que o agrupamento apenas reproduza a divisão administrativa. Valores ausentes viram uma categoria própria "ausente" somente na matriz de ML, sem alterar o dado tratado.

Requisitos atendidos: RF11, RIA02, RIA03, RIA04

Etapa 7 – Normalização e agrupamento

Atributos numéricos, se houver, são padronizados com StandardScaler. O agrupamento usa o **K-Means** do scikit-learn (inicialização k-means++, número fixo de inicializações e semente aleatória registrada). A execução atribui um perfil a cada registro elegível. O pipeline roda por linha de comando, independente do site, e pode ser testado separadamente.

Requisitos atendidos: RF12, RIA05, RIA10

Etapa 8 – Avaliação e escolha de k

O sistema executa varreduras com k de 2 a 10 e registra inércia (método do cotovelo), coeficiente de silhueta e índice Davies-Bouldin. A silhueta é calculada sobre uma amostra aleatória com semente fixa, para evitar custo quadrático. A estabilidade é verificada comparando agrupamentos de diferentes sementes pelo índice de Rand ajustado. Essas informações apoiam a equipe; a escolha final de k é decisão humana, documentada, e não uma decisão automática da IA.

Requisitos atendidos: RIA06, RIA07

Etapa 9 – Perfis e rastreabilidade

Para cada perfil, o sistema calcula tamanho, percentuais de LED, telegestão e iluminação de destaque, distribuição de objetos iluminados e de tipos de logradouro e distribuição por regional e bairro. Os perfis recebem nomes neutros (Perfil 1, Perfil 2, ...) e a interface descreve apenas o que os números mostram. Cada execução grava a carga de dados usada (com hash), os atributos, os parâmetros, a semente, as versões das bibliotecas e as métricas, o que permite reproduzir a análise.

Requisitos atendidos: RF14, RF15, RIA08, RIA09, RIA10, RNF06

Etapa 10 – Aplicação web

Interface em HTML, CSS e JavaScript com três áreas: **Mapa** (Leaflet; unidades como pontos, trechos como linhas, camadas ativáveis, seleção de elemento com seus atributos e coloração por perfil), **Relatórios** (indicadores, comparação entre regionais e bairros com gráficos Plotly, relatório de qualidade e exportação) e **Análise IA** (perfis, métricas de avaliação e parâmetros da execução). Uma nota fixa informa que perfis descrevem padrões nos dados e não qualidade. As bibliotecas JavaScript ficam empacotadas localmente. O mapa-base externo (OpenStreetMap) é opcional: sem conexão, o mapa continua funcionando com as camadas do próprio projeto.

Requisitos atendidos: RF05, RF06, RF07, RF08, RF09, RF13, RNF01, RNF04

4. Técnicas e algoritmos de IA escolhidos

Problema de ML. Identificar automaticamente perfis semelhantes de infraestrutura de iluminação pública. É um problema de aprendizado não supervisionado por agrupamento (clustering), pois as bases não têm um rótulo de interesse (defeito, qualidade) que sustente aprendizado supervisionado.

4.1 Atributos e preparação

Item	Decisão	Justificativa
Unidade de análise	Registro válido da base UNIDADE_ILUMINACAO_PUBLICA	RIA02; é a base com os atributos de infraestrutura
Atributos do modelo	LED, telegestão, iluminação de destaque, objeto iluminado, tipo de logradouro; opcional: densidade local	Descrevem a infraestrutura em si (RIA03)
Fora do modelo	Identificadores, nome do logradouro, coordenadas, bairro e regional	Identificadores não têm significado analítico; bairro e regional servem para interpretar os perfis
Codificação	One-hot dos categóricos; categoria própria "ausente" só na matriz de ML	RIA04; não altera o dado tratado (RD05)
Escala	StandardScaler (e log1p) em atributos numéricos	K-Means usa distância euclidiana (RF11)
Categorias raras	Agrupadas em "outros" apenas na matriz de ML, com limite documentado	Evita colunas quase vazias

4.2 Algoritmo: K-Means

O algoritmo escolhido é o **K-Means** (scikit-learn, k-means++). Justificativa técnica (RIA05): é simples e bem estabelecido; escala bem para a quantidade de unidades esperada; seus centroides são fáceis de interpretar, pois com atributos codificados em one-hot cada centroide expressa a proporção de cada categoria no perfil; roda em CPU, sem custo (RNF02); e é reprodutível com semente fixa (RIA10). Como os atributos são em grande parte binários, o K-Means serve como linha de base interpretável, e essa limitação será registrada na documentação.

Alternativa	Situação	Motivo
DBSCAN / HDBSCAN	Não adotada	Dados com muitos registros idênticos e poucas dimensões contínuas; densidade pouco informativa
Hierárquico aglomerativo	Não adotada	Custo quadrático em memória para a quantidade de unidades
Gaussian Mixture	Não adotada	Pressupõe distribuições contínuas; atributos são binários
K-Modes / K-Prototypes	Evolução possível	Mais adequados a categóricos, mas exigem biblioteca adicional; só se o K-Means for insuficiente
Deep learning	Fora do escopo	Sem necessidade para o problema (Lista de Requisitos, Seção 10)

4.3 Avaliação, escolha de k e interpretação

- **Métricas:** inércia (cotovelo), coeficiente de silhueta (em amostra com semente fixa) e índice Davies-Bouldin, calculados para cada k de 2 a 10 e registrados (RIA06, RIA07).
- **Estabilidade:** índice de Rand ajustado entre execuções com sementes diferentes, para saber se os perfis dependem da inicialização.
- **Escolha de k:** feita pela equipe a partir das métricas e da interpretabilidade, com a justificativa registrada. Não é apresentada como decisão automática.
- **Interpretação:** estatísticas e distribuições por perfil conferíveis a partir dos registros do grupo (RF14, RIA08). Nenhum perfil recebe rótulo de qualidade.
- **Visualização auxiliar:** projeção em 2D (PCA) apenas para ilustrar a separação entre perfis, com aviso de que é uma simplificação.

4.4 Reprodutibilidade

Cada execução grava o hash da carga de dados, os atributos, a codificação e a normalização, o algoritmo e seus parâmetros, a semente, as versões das bibliotecas e as métricas (RF15, RIA09, RIA10). Duas execuções com a mesma configuração devem produzir atribuições equivalentes, e esse comportamento será coberto por teste.

5. Papel da base do Portal de Dados Abertos na solução

As duas bases do Portal de Dados Abertos da PBH são a única fonte de informação do sistema (RD01). Elas sustentam todas as funcionalidades, da visualização ao modelo de IA.

Base	Conteúdo usado	Onde entra na solução
UNIDADE_ILUMINACAO_PUBLICA (pontos)	Identificação, iluminação de destaque, objeto iluminado, LED, telegestão, tipo e nome do logradouro, bairro, regional, geometria	Camada de pontos no mapa; filtros; indicadores gerais, por regional e por bairro; **única base do modelo de IA** (atributos de infraestrutura); bairro e regional para interpretar os perfis
TRECHO_ILUM_PUBLICA (linhas)	Identificação do trecho, identificação da base, indicador disponível, geometria linear	Camada de linhas no mapa e consulta de atributos; contexto geográfico complementar; não entra como atributo do modelo no MVP (RIA02)

Limites dos dados. As bases não informam falhas, consumo, potência, data de instalação nem estado de conservação. Por isso o sistema não faz inferências sobre defeitos ou qualidade. Os valores dos indicadores só existirão depois do processamento real, e valores ausentes ou categorias desconhecidas são tratados conforme as definições da fonte (RD05). Os dados são tratados como possivelmente incompletos e a completude é reportada no relatório de qualidade.

6. Modelo de dados (SQLite)

Esquema lógico inicial. Os nomes definitivos das colunas seguirão o dicionário de dados publicado pela fonte.

Tabela	Campos principais	Finalidade
unidade_iluminacao	id, número de identificação, destaque, objeto, LED, telegestão, tipo e nome do logradouro, bairro, regional, lat, lon, geometria (WKT)	Dados tratados das unidades
trecho_iluminacao	id, id da base, indicador, geometria (WKT), caixa envolvente	Dados tratados dos trechos
carga	id, base, arquivo, hash SHA-256, data, lidos, aceitos, rejeitados	Rastreio e integridade (RNF07)
ocorrencia_qualidade	id da carga, tipo, campo, id do registro, detalhe	Relatório de qualidade (RF10, RD06)
execucao_clustering	id, id da carga, atributos, parâmetros, semente, versões, métricas, data	Rastreabilidade (RF15, RIA09)
atribuicao_perfil	id da execução, id da unidade, perfil	Perfil de cada unidade (RF13)
perfil_estatistica	id da execução, perfil, tamanho, estatísticas	Caracterização (RF14)

7. Tecnologias e implantação local

Camada	Tecnologia	Observação
Linguagem	Python 3	Ingestão, tratamento, ML e API
Dados e geometrias	pandas, GeoPandas, Shapely	Leitura, validação e transformação de CRS
Machine Learning	scikit-learn	Codificação, normalização, K-Means e métricas
Banco	SQLite	PostgreSQL/PostGIS apenas se necessário
Backend	FastAPI, Uvicorn	Servidor local
Frontend	HTML, CSS, JavaScript, Leaflet, Plotly	Bibliotecas empacotadas localmente
Testes	pytest	Regras de validação, indicadores e reprodutibilidade (RNF11)
Versionamento	Git/GitHub	Código e documentação

O sistema roda em uma única máquina, sem GPU, sem nuvem e sem serviços pagos (RNF01, RNF02, RNF09). O código é organizado em módulos separados – ingestão, tratamento, armazenamento, API, pipeline de ML e

interface – com responsabilidades documentadas (RNF10). Os tempos das principais operações serão medidos e registrados (RNF03), e a instalação e a execução serão descritas em documentação verificada por um integrante que não participou da implementação (RNF05, RNF12).

8. Rastreabilidade entre requisitos e arquitetura

Etapa / componente	Requisitos atendidos
1 Fonte de dados	RD01, RD02, RD03
2 Ingestão	RF01
3 Validação, limpeza e padronização	RF02, RF03, RF10, RD04, RD05, RD06
4 Armazenamento	RF04, RNF07
5 API / Backend	RF06, RF07, RF08, RF09, RF13
6 Engenharia de atributos	RF11, RIA02, RIA03, RIA04
7 Normalização e agrupamento	RF12, RIA01, RIA05
8 Avaliação e escolha de k	RIA06, RIA07
9 Perfis e rastreabilidade	RF14, RF15, RIA08, RIA09, RIA10
10 Aplicação web	RF05, RF06, RF07, RF08, RF09, RF13, RNF04
Transversais	RNF01 (operação local), RNF02 (custo), RNF03 (desempenho), RNF05 (portabilidade), RNF06 (reprodutibilidade), RNF08 (erros), RNF09 (segurança local), RNF10 (manutenibilidade), RNF11 (testes), RNF12 (documentação)

O fluxo de aceitação da Lista de Requisitos (Importação, Validação, Limpeza, Armazenamento, Visualização, Filtragem, Indicadores, Preparação para IA, Agrupamento, Avaliação, Interpretação e Registro da execução) corresponde exatamente ao caminho das etapas 2 a 10 da Figura 1.

9. Alinhamento ao Manifesto Ético do Time

- **Transparência e explicabilidade:** K-Means com atributos interpretáveis, parâmetros, métricas e atributos sempre visíveis; nada de caixa-preta.
- **Justiça e mitigação de viés:** o sistema não usa dados de pessoas, mas pode haver viés de cobertura (regiões com cadastro menos completo). O relatório de qualidade expõe a completude por regional, e os perfis não são lidos como mérito ou deficiência de uma região.
- **Privacidade e segurança:** apenas dados públicos e não pessoais, processados localmente e sem envio a terceiros (RNF09).
- **Supervisão humana:** a IA apoia a análise. Escolha de k, interpretação dos perfis e qualquer uso dos resultados são decisões humanas; o sistema não decide nada sobre a infraestrutura.
- **Sustentabilidade:** modelo leve, em CPU, treinado em minutos, sem GPU nem nuvem.

10. Fases de desenvolvimento, riscos e limitações

O desenvolvimento seguirá dez fases sequenciais, e uma fase só é considerada concluída quando a anterior está funcionando, para chegar a 06/12/2026 com o fluxo completo demonstrável.

Fase	Atividade	Etapas da Figura 1
1	Carregar dados	1, 2
2	Limpar e validar	3
3	Banco de dados	4
4	Mapa	5, 10
5	Filtros e relatórios	5, 10
6	Engenharia de atributos	6
7	Agrupamento e avaliação	7, 8

Fase	Atividade	Etapas da Figura 1
8	Integrar perfis ao mapa	9, 10
9	Testes	Todas
10	Documentação	Todas

Risco ou limitação	Mitigação
Atributos majoritariamente categóricos limitam o K-Means	Registrar a limitação; avaliar K-Modes/K-Prototypes só se necessário
Volume alto de pontos no mapa	GeoJSON por área visível, zoom adaptativo e medição de tempo (RNF03)
Formato ou CRS dos arquivos diferente do esperado	Verificação na ingestão e testes com amostra controlada
Perfis interpretados como qualidade	Nomes neutros, nota fixa na interface e documentação
Prazo curto (timeboxing)	Escopo mínimo, fases sequenciais e itens fora do escopo explícitos
Mapa-base sem internet	Mapa-base opcional; camadas próprias funcionam offline

Alterações nesta arquitetura serão feitas de forma controlada e documentadas, mantendo a compatibilidade com a Lista de Requisitos e com o prazo final.